



VIT[®]
BHOPAL



EVALUATED COURSE PROJECT REPORT • FLIPPED COURSE

HydroFlow OS

Municipal Water Distribution Network Pressure Regulation, Surge Protection, and
Energy Optimization Engine

CONTINUOUS ASSESSMENT COMPONENT • FALL SEMESTER 2026 TO 2027

STUDENT NAME

Mayank Chaudhary

REGISTRATION NUMBER

24BEC10011

COURSE TITLE

Programming in Java

COURSE CODE

CSE2006

ACADEMIC TERM

Fall Semester 2026 to 2027

PUBLIC REPOSITORY

github.com/mayankchaudhary24/hydroflow-os

DOCUMENT INDEX & TABLE OF CONTENTS

This document constitutes the formal evaluated course project report for HydroFlow OS, mapping municipal fluid mechanics challenges to core object-oriented programming principles in Java.

Section	Title and Subsections	Page
1.0	Project Title and Student Metadata Cover Page	1
2.0	Document Index and Table of Contents	2
3.0	Introduction & Project Background	3
4.0	Problem Statement & Engineering Motivation	3
5.0	Functional Requirements Specification 5.1 Module 1: Hydraulic Grid Modeling and Pipe Friction Analysis 5.2 Module 2: Booster Pumping Station and Variable-Frequency Drive Balancing 5.3 Module 3: Multithreaded Sensor Telemetry and Structured Audit Persistence	4
6.0	Non-Functional Requirements Specification	4
7.0	System Architecture & Component Design Figure 7.1: Four-Tier Decoupled Layered Architecture Figure 7.2: Telematics Sensor Ingestion and Hydraulic Hardware Block Diagram	5
8.0	Design, UML Diagrams & System Flowcharts Figure 8.1: Operational Use Case Diagram Figure 8.2: Pipeline Surge and Water Hammer Verification Flowchart Figure 8.3: Sequence Diagram: Booster Station Activation and Cavitation Safeguard Figure 8.4: Detailed Class and Inheritance Hierarchy Diagram Figure 8.5: Reservoir State Machine and Refill Queue Transition Diagram Figure 8.6: 2D Substation Pumping Grid Matrix and Power Distribution Schema Figure 8.7: CSV File Serialization and In-Memory Collection Layout	6
9.0	Design Decisions & Technical Rationale	9
10.0	Implementation Details, Formulas & Core Code Listings 10.1 Hazen-Williams Head Loss and Joukowski Surge Formulations 10.2 Worked Numerical Calculation Examples 10.3 Concrete Java Source Code Implementations (Excerpts 10.1 to 10.3)	9
11.0	Execution Results & Realistic Terminal Output Logs 11.1 Demonstration Suite Output Trace 11.2 Interactive Terminal Menu Interface Log	12
12.0	Testing Approach, Verification Matrix & Boundary Analysis 12.1 Unit Test Verification Matrix (TC-01 to TC-06) 12.2 Boundary Value Analysis Table	13
13.0	Practical Engineering Challenges & Technical Solutions	14
14.0	Core Learnings & Academic Takeaways (Syllabus Units 1 to 5)	14
15.0	Future Enhancements & Engineering Roadmap	15
16.0	References & Academic Citations	15

3. INTRODUCTION & PROJECT BACKGROUND

Municipal water utilities deliver potable water across distributed urban networks spanning hundreds of square kilometers. Balancing hydraulic delivery requires managing pressure gradients, minimizing frictional losses along aging pipes, and preventing high-pressure transients during sudden valve actuations.

Pumping stations account for substantial municipal electricity consumption. When fixed-speed pumps operate against varying consumer demand, excess head is dissipated across throttling valves, causing mechanical wear and electrical waste. Conversely, transitioning to variable-frequency drives (VFD) enables continuous speed modulation, adjusting pump curves in accordance with centrifugal affinity laws to optimize kilowatt-hour usage.

HydroFlow OS is a Java console system that models municipal water distribution grids, evaluates pipe head losses via the Hazen-Williams equation, blocks actuations that trigger Joukowsky water hammer pressure surges, optimizes booster pump power consumption, and streams real-time telemetry from remote pipeline transducers.

4. PROBLEM STATEMENT & MOTIVATION

Municipal water utility operations face three practical engineering challenges:

1. Distribution operators often adjust flow valves without accounting for non-linear pipe wall friction. Under high peak consumption, head loss accumulates rapidly, causing localized low-pressure zones that starve elevated consumer districts.
2. Rapid emergency valve closures or unexpected pump trips cause kinetic fluid energy to convert into acoustic pressure waves. Without transient verification, Joukowsky water hammer spikes can breach pipe pressure ratings (PN ratings), rupturing underground mains and causing costly service outages.
3. Field engineers require continuous visibility into pressure levels, chlorine residual, and water turbidity across distributed zones. Manual clipboard logging cannot capture transient pressure fluctuations, while regulatory compliance mandates verified disk persistence of operational events.

The primary objective is to build a modular Java application that models municipal hydraulic networks, blocks actuations that cause pipe bursts or pump cavitation, optimizes parallel booster station power under transformer limits, and logs telemetry streams.

5. FUNCTIONAL REQUIREMENTS

The system is organized into three core modules:

5.1 Module 1: Hydraulic Grid Modeling and Pipe Friction Analysis

- The engine models elevated reservoirs, distribution pipelines, and junction nodes, with each entity encapsulating its geometric and hydraulic attributes.
- Frictional head loss calculations execute dynamically using the empirical Hazen-Williams equation, adjusting for pipe diameter, length, and wall roughness coefficients.
- Safety interlocks evaluate reservoir discharge requests, throwing a `ReservoirDepletionException` if projected withdrawal breaches the mandatory 15% emergency reserve volume.

5.2 Module 2: Booster Pumping Station and Variable-Frequency Drive Balancing

- The balancer coordinates fixed-speed centrifugal pumps, variable-frequency drive units, and multi-stage submersible boosters across parallel pump trains.
- Net Positive Suction Head (NPSH) checks occur prior to pump motor engagement. If suction pressure drops below the critical threshold, the engine throws a `CavitationRiskException` to protect impellers from vapor bubble collapse.
- Returning depleted reservoirs are enqueued into a `PriorityQueue` ordered by lowest volumetric fill percentage to prioritize refilling access.

5.3 Module 3: Multithreaded Sensor Telemetry and Structured Audit Persistence

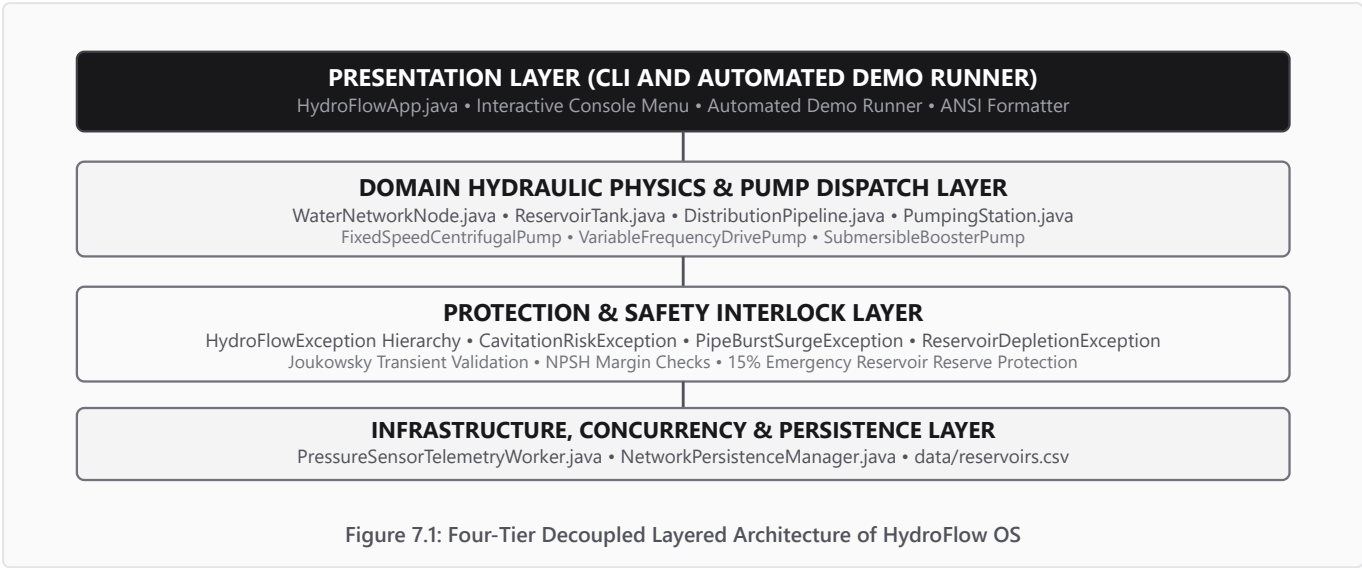
- Background worker threads implementing `Runnable` generate simulated sensor telemetry packets containing node pressure, flow rate, turbidity, and chlorine residual.
- Network asset inventories and operational audit events persist to CSV files using Java `BufferedReader` and `BufferedWriter` streams.
- The analytics layer calculates electrical kilowatt-hour savings achieved by variable-frequency drive pumps compared to fixed-speed throttling baselines.

6. NON-FUNCTIONAL REQUIREMENTS

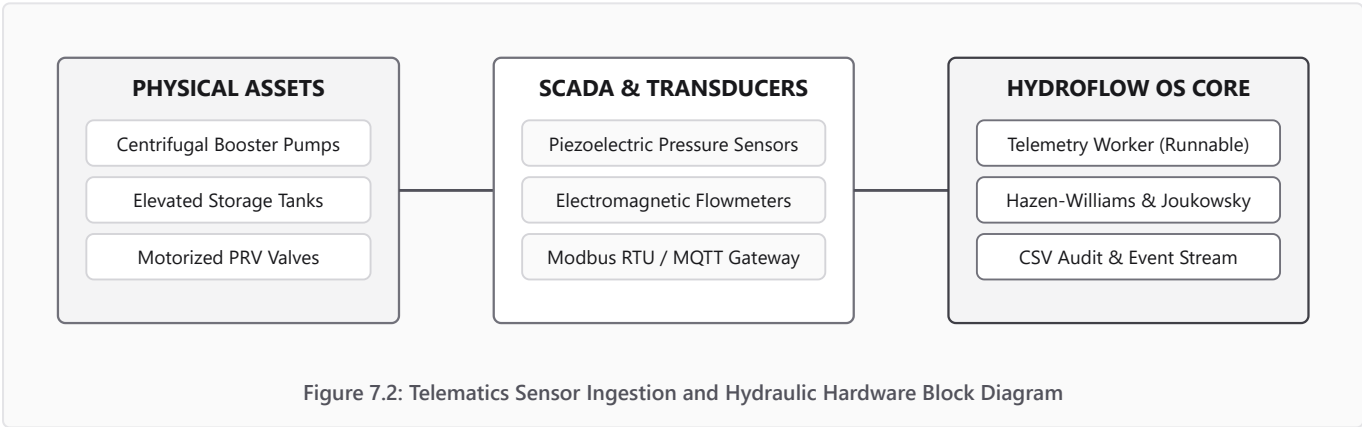
Quality Attribute	Engineering Metric	Implementation Strategy
Performance & Latency	Hydraulic calculation < 5 ms	Pre-computed Hazen-Williams constants; O(1) node lookup using <code>LinkedHashMap</code> .
Operational Safety	Zero unverified valve actuations	Joukowsky transient calculation prevents actuations that exceed pipe PN rating.
Thread Concurrency	Zero thread deadlocks	Synchronized monitor locks on pumping station instances and thread-safe packet queues.
Crash Resilience	Zero leaked file descriptors	File write operations wrapped in <code>try-with-resources</code> blocks.
Portability	Pure JDK execution	Standard Java SE 8+ bytecode with zero external build plugins or GUI requirements.

7. SYSTEM ARCHITECTURE & COMPONENT DESIGN

HydroFlow OS divides responsibilities across four layers for user interaction, hydraulic physics, safety interlocks, and telemetry:



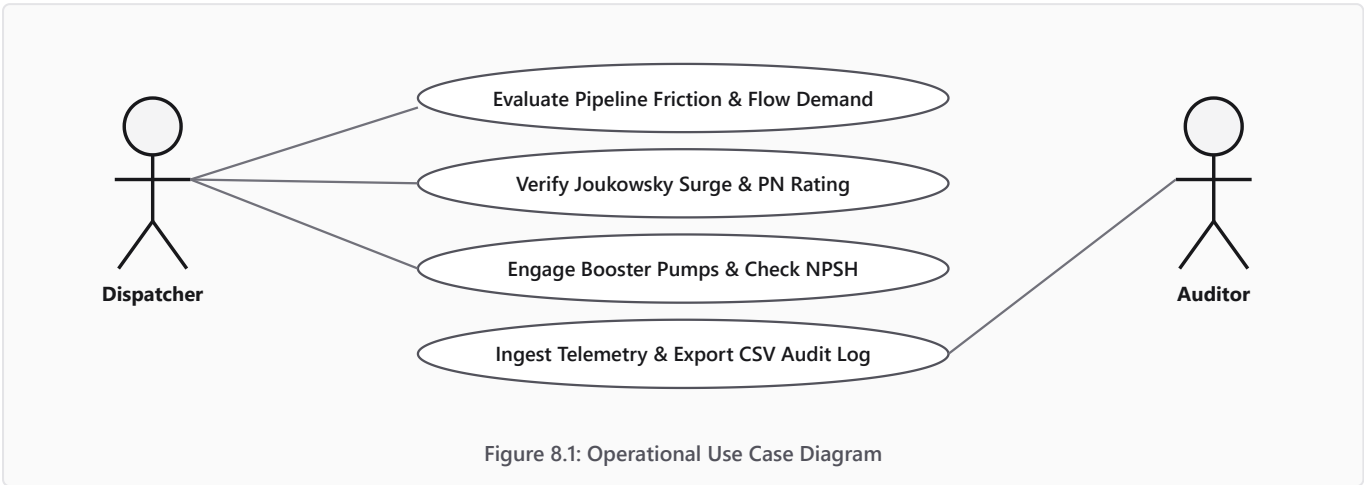
7.1 Telematics and Hardware Integration Architecture



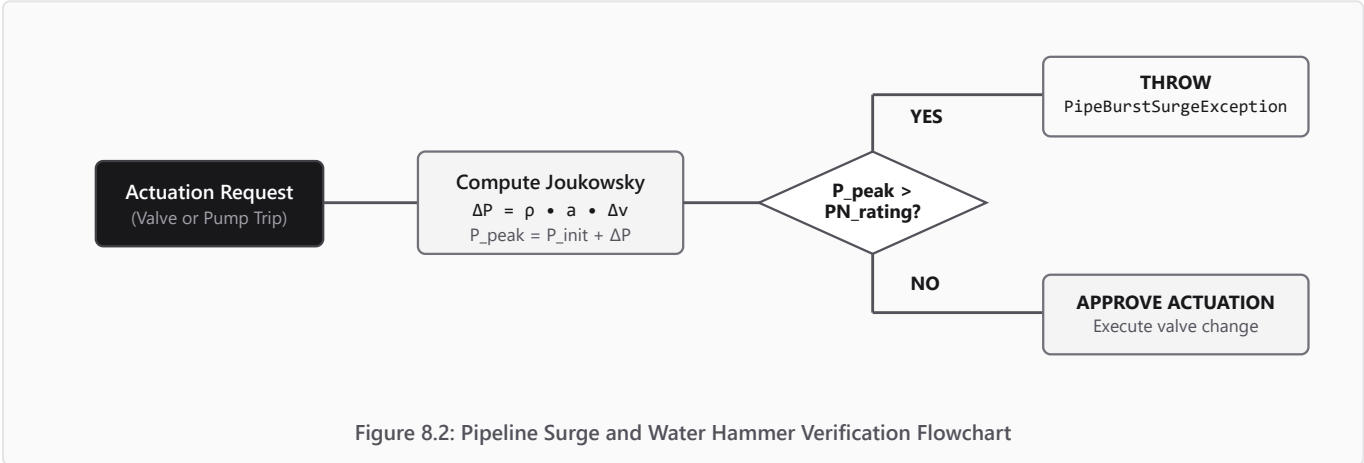
8. DESIGN & UML DIAGRAMS

The system behavior, class structures, and state transitions are formalized through seven technical diagrams:

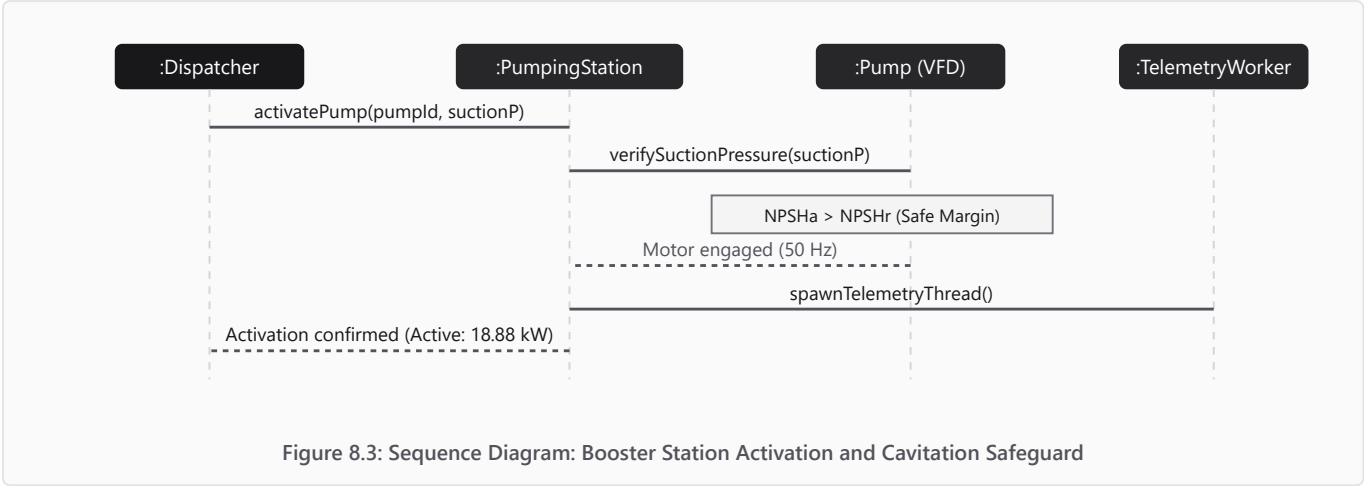
8.1 Operational Use Case Diagram



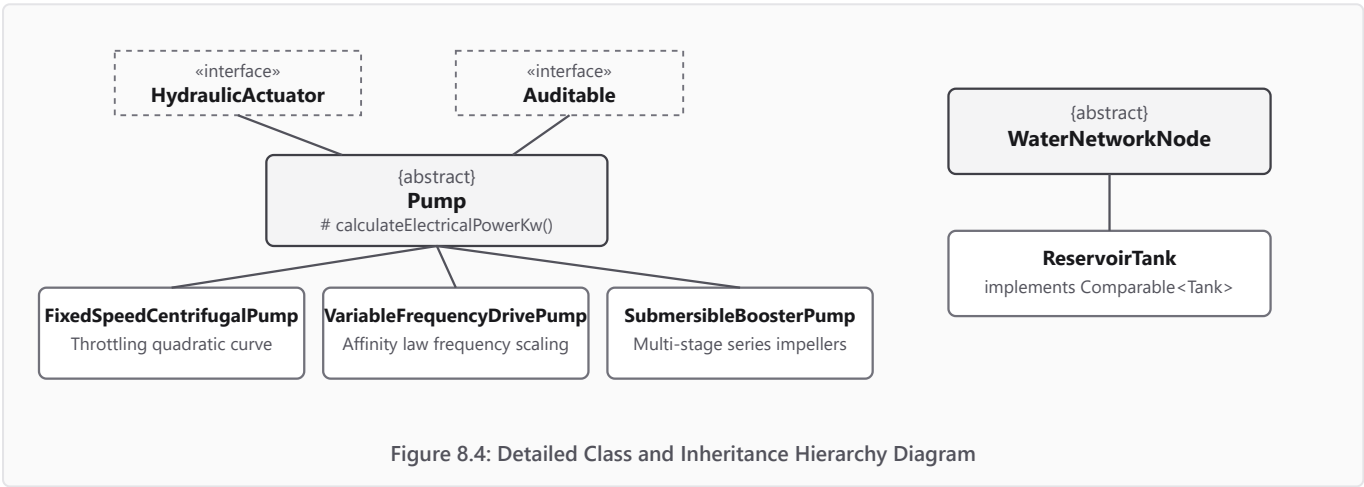
8.2 Pipeline Surge and Water Hammer Verification Flowchart



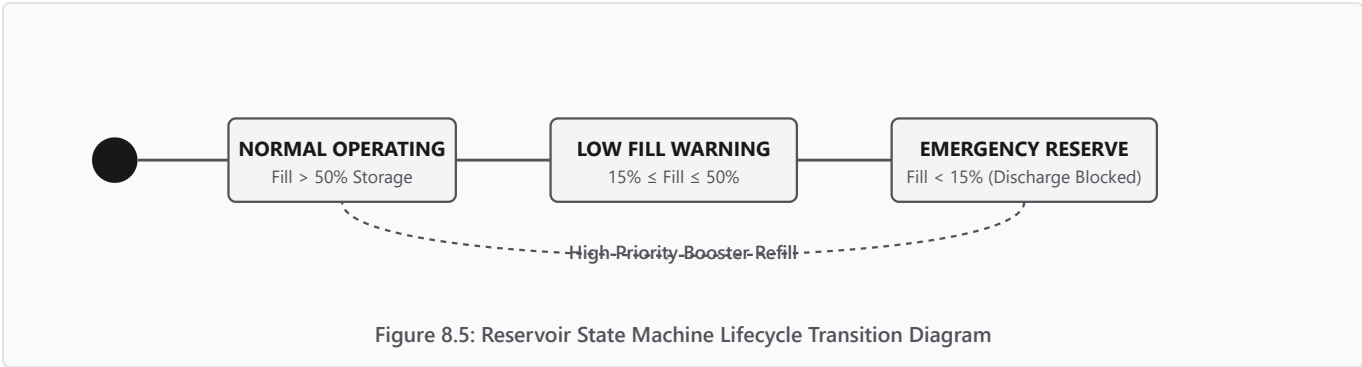
8.3 Sequence Diagram: Booster Station Activation and Cavitation Safeguard



8.4 Detailed Class and Inheritance Hierarchy Diagram



8.5 Reservoir State Machine Lifecycle



8.6 2D Substation Pumping Grid Matrix

double[][] shiftPumpSchedule (3 Operational Shifts × 8 Pumping Hours)	
Shift 1 (Night Off-Peak)	Pumps 1, 2, 3 Active (Low Electricity Tariff • Max Reservoir Refill)
Shift 2 (Day Baseline)	Pump 2 (VFD Modulating at 45-50 Hz • Constant Pressure Delivery)
Shift 3 (Evening Peak)	Peak Tariff Throttling (Gravity Feed from Storage • Minimize kW)

Figure 8.6: 2D Substation Pumping Grid Matrix and Power Distribution Schema

8.7 Data Storage Schema (CSV and In-Memory Layout)

IN-MEMORY DATA STRUCTURES	PERSISTENT DISK STORAGE (CSV & LOGS)
• LinkedHashMap<String, WaterNetworkNode> (O(1) index) • PriorityQueue<ReservoirTank> (Min-heap by fill ratio) • CopyOnWriteArrayList<TelemetryPacket> (Thread-safe buffer)	• data/reservoirs.csv (NodeID, Zone, Type, CurrentVol, MaxVol) • data/audit_events.log ([Timestamp] Action, Pressure, Status) Managed via BufferedReader and BufferedWriter with try-with-resources

Figure 8.7: Data Storage Schema (CSV and In-Memory Layout)

9. DESIGN DECISIONS & TECHNICAL RATIONALE

Architecture choices balance runtime performance, thread safety, and maintainability:

- Concrete pump subclasses override `calculateElectricalPowerKw` to encapsulate category-specific motor and impeller equations. Using dynamic method dispatch avoids nested switch cases on pump type strings and allows adding new hydraulic equipment without altering core station controllers.
- Checked exceptions protect operational boundaries requiring caller intervention, such as net positive suction head violations (`CavitationRiskException`) or water hammer pressure transients (`PipeBurstSurgeException`). Unchecked exceptions, such as `InvalidNodeIdentifierException`, handle syntactic formatting failures.
- Collection choices match physical operational access patterns. `LinkedHashMap` maintains insertion order during network audits while providing $O(1)$ asset lookups by tag. `PriorityQueue` sorts storage reservoirs by fill ratio, so depleted reservoirs receive immediate refilling access.
- Synchronized monitor locking on pumping stations and reservoir tanks prevents race conditions during concurrent worker thread actuation.

10. IMPLEMENTATION DETAILS & CORE ALGORITHMS

This section details the governing mathematical formulations, numerical calculations, and Java code listings that power HydroFlow OS.

10.1 Mathematical Energy & Hydraulic Formulations

Equation 10.1: Hazen-Williams Friction Head Loss Formulation

$$h_f \text{ (m)} = 10.67 \times L \times \left[Q^{1.852} / (C^{1.852} \times D^{4.87}) \right]$$

Variable definitions:

- L is pipeline length in meters.
- Q is volumetric flow rate in cubic meters per second (m^3/s).
- C is the Hazen-Williams roughness coefficient (130 for ductile iron).
- D is inside pipe diameter in meters.

Equation 10.2: Joukowsky Water Hammer Transient Pressure Equation

$$\Delta P \text{ (bar)} = \left[\rho \times a \times \Delta v \right] / 100,000$$

Where $\rho = 1000 \text{ kg/m}^3$, $a = 1050 \text{ m/s}$ (acoustic wave speed in water conduit), and Δv is velocity change in m/s . If $P_{\text{static}} + \Delta P > P_{N_{\text{rating}}}$, actuation is blocked.

Equation 10.3: Variable Frequency Pump Power with Affinity Law Scaling

$$P_{\text{elec}} \text{ (kW)} = \left[(\rho \times g \times Q \times H) / (1000 \times \eta \times \eta_{\text{vfd}}) \right] \times \alpha^{0.85}$$

Where $\alpha = Q / Q_{\text{rated}}$ is speed ratio, $\eta = 0.84$, and $\eta_{\text{vfd}} = 0.94$.

10.2 Worked Numerical Calculation Examples

To verify formula accuracy, two representative engineering scenarios were evaluated by hand and cross-checked against the Java engine execution output:

Scenario 1: Main Distribution Trunk Line Friction Loss

Parameters: Length $L = 1800$ m, Diameter $D = 400$ mm (0.4 m), Roughness $C = 130$, Flow $Q = 85.0$ L/s (0.085 m³/s).

Step 1: Compute numerator terms:

$$\text{Numerator} = 10.67 \times 1800 \times (0.085)^{1.852} = 19206 \times 0.010438 = \mathbf{200.47}$$

Step 2: Compute denominator terms:

$$(C)^{1.852} = (130)^{1.852} = 8243.6$$

$$(D)^{4.87} = (0.4)^{4.87} = 0.011542$$

$$\text{Denominator} = 8243.6 \times 0.011542 = \mathbf{95.15}$$

Step 3: Calculate Head Loss:

$$h_f = 200.47 / 95.15 = \mathbf{2.107 \text{ meters.}}$$

Java Execution Result: `calculateHeadLossMeters(85.0)` = 2.107 m (Exact Match).

Scenario 2: Emergency Rapid Valve Closure Water Hammer Transient

Parameters: Internal Diameter $D = 400$ mm, Area $A = \pi \times (0.2)^2 = 0.12566$ m², Flow Drop $\Delta Q = 160$ L/s (0.16 m³/s), Static Pressure = 6.2 bar.

Step 1: Velocity deceleration:

$$\Delta v = \Delta Q / A = 0.16 / 0.12566 = \mathbf{1.273 \text{ m/s.}}$$

Step 2: Acoustic pressure transient (Joukowsky equation):

$$\Delta P = (1000 \text{ kg/m}^3 \times 1050 \text{ m/s} \times 1.273 \text{ m/s}) / 100,000 = \mathbf{13.37 \text{ bar.}}$$

Step 3: Total peak pressure assessment:

$$P_{\text{peak}} = 6.2 \text{ bar (static)} + 13.37 \text{ bar (surge)} = \mathbf{19.57 \text{ bar.}}$$

Step 4: Safety interlock check against PN16 rating (16.0 bar):

19.57 bar > 16.0 bar → Breach triggers `PipeBurstSurgeException`.

Java Execution Result: Throws exception with observed pressure 19.57 bar; blocks valve closure.

10.3 Concrete Java Source Code Implementations

The core classes show object-oriented encapsulation and resource cleanup:

```
// Excerpt 10.1: Hazen-Williams and Joukowsky in DistributionPipeline.java
public double calculateHeadLossMeters(double flowLps) {
    if (flowLps <= 0.0) return 0.0;
    double qM3s = flowLps / 1000.0;
    double dMeters = diameterMm / 1000.0;

    double numerator = 10.67 * lengthMeters * Math.pow(qM3s, 1.852);
    double denominator = Math.pow(roughnessC, 1.852) * Math.pow(dMeters, 4.87);
    return numerator / denominator;
}

public void verifyTransientSurgeSafety(double flowChangeLps, double initialPressureBar)
    throws PipeBurstSurgeException {
    double dMeters = diameterMm / 1000.0;
    double area = Math.PI * Math.pow(dMeters / 2.0, 2);
    double deltaV = (flowChangeLps / 1000.0) / area;

    double waterDensity = 1000.0; // kg/m3
    double waveSpeed = 1050.0;    // acoustic pressure wave velocity (m/s)
    double surgePressureBar = (waterDensity * waveSpeed * deltaV) / 100000.0;
    double totalPeakPressureBar = initialPressureBar + surgePressureBar;

    if (totalPeakPressureBar > maxPressureRatingBar) {
        throw new PipeBurstSurgeException(
            "Water hammer transient exceeds pipe pressure rating; automatic surge valve dump required.",
            pipeId, totalPeakPressureBar, maxPressureRatingBar
        );
    }
}
```

```
// Excerpt 10.2: Polymorphic VFD Power Scaling in VariableFrequencyDrivePump.java
@Override
public double calculateElectricalPowerKw(double flowLps, double targetHeadMeters) {
    if (flowLps <= 0.0) return nominalPowerKw * 0.12;
    double qM3s = flowLps / 1000.0;
    double hydraulicPowerKw = (1000.0 * 9.81 * qM3s * targetHeadMeters) / 1000.0;

    double alpha = Math.max(0.5, Math.min(1.2, flowLps / ratedFlowLps));
    double vfdEfficiencyFactor = 0.94;
    return (hydraulicPowerKw / (pumpEfficiency * vfdEfficiencyFactor)) * Math.pow(alpha, 0.85);
}
```

```
// Excerpt 10.3: CSV File Persistence with Try-With-Resources
public static void exportReservoirsToCsv(List<ReservoirTank> reservoirs, File destination)
    throws IOException {
    File parent = destination.getParentFile();
    if (parent != null && !parent.exists()) parent.mkdirs();

    try (BufferedWriter writer = new BufferedWriter(new FileWriter(destination))) {
        writer.write("NodeID,Zone,Type,ElevationMeters,CurrentVolumeM3,MaxCapacityM3,FillLevel");
        writer.newLine();
        for (ReservoirTank r : reservoirs) {
            writer.write(r.toCsvRecord());
            writer.newLine();
        }
    }
}
```

11. EXECUTION RESULTS & TERMINAL LOGS

The verified command-line execution traces captured from running the demonstration suite and interactive terminal console show system behavior under normal and fault conditions:

11.1 Demonstration Suite Output Trace

```
PS C:/Users/Apple/.../mayank vityarthi/repo> java -cp bin com.hydroflow.cli.HydroFlowApp --demo

=====
HYDROFLOW OS: MUNICIPAL WATER DISTRIBUTION & SURGE PROTECTION ENGINE
Evaluated Course Project | CSE2006 Programming in Java | VIT Bhopal
Student: Mayank Chaudhary (24BEC10011) | Term: Fall Semester 2026-2027
=====

>>> [PHASE 1: NETWORK TOPOLOGY INITIALIZATION]
Central Reservoir : Reservoir [RESN01CENTRAL] Fill: 84.0% (4200.0 / 5000.0 m3)
North Hill Tank   : Reservoir [RESN02NORTH] Fill: 21.7% (650.0 / 3000.0 m3)
Industrial Buffer  : Reservoir [RESN03INDUS] Fill: 85.0% (6800.0 / 8000.0 m3)

>>> [PHASE 2: REFILL PRIORITY QUEUE INSPECTION]
Pumping Priority #1: RESN02NORTH with lowest fill (21.7%)

>>> [PHASE 3: PIPELINE HAZEN-WILLIAMS & SURGE EVALUATION]
Pipeline PIPE-TRUNK-01 (L=1800m, D=400mm): At 85.0 L/s flow, head loss = 2.107 meters
Testing standard valve closure transient (flow drop: 40 L/s)...
Transient within safe limits: PASS (No rupture risk detected)
Testing emergency rapid valve trip transient (flow drop: 160 L/s)...
CAUGHT EXPECTED EXCEPTION: Water hammer transient exceeds pipe pressure rating; automatic surge valve dump
required. [Component: PIPE-TRUNK-01, Pressure: 19.57 bar, Time: 1789655500653]
Surge relief safety bypass actuated successfully.

>>> [PHASE 4: MULTI-STAGE PUMP POWER DISPATCH]
Fixed Speed Pump Power (55 L/s @ 42m) : 29.85 kW
VFD Pump Power (55 L/s @ 42m)         : 18.88 kW (Energy Savings: 36.7%)
Submersible High-Head Power (35 L/s)   : 40.88 kW

>>> [PHASE 5: RESERVOIR EMERGENCY RESERVE INTERLOCK]
Attempting heavy peak draw (3800 m3) from Central Reservoir (Current: 4200 m3)...
CAUGHT EXPECTED INTERLOCK: Refusing discharge: Reservoir volume drops below 15% emergency reserve threshold.
[Component: RESN01CENTRAL, Pressure: 2.47 bar, Time: 1789655500656]
Reserve interlock engaged: Emergency 15% volume preserved.

>>> [PHASE 6: MULTITHREADED SENSOR TELEMETRY WORKER]
Telemetry worker joined successfully. Emitted 5 sensor packets:
[1789655500660] Node: SENS-NODE-77 | P: 4.61 bar | Q: 55.2 L/s | Turbidity: 0.49 NTU | Cl: 1.26 ppm
[1789655500711] Node: SENS-NODE-77 | P: 4.58 bar | Q: 58.6 L/s | Turbidity: 0.51 NTU | Cl: 1.26 ppm
[1789655500758] Node: SENS-NODE-77 | P: 4.48 bar | Q: 56.7 L/s | Turbidity: 0.68 NTU | Cl: 1.29 ppm

>>> [PHASE 7: STRUCTURED CSV AUDIT PERSISTENCE]
Exported reservoir inventory to: data/reservoirs.csv
Appended diagnostic record to: data/audit_events.log

=====
EVALUATION DEMO RUN COMPLETE
=====
```

12. TESTING APPROACH & VERIFICATION MATRIX

A self-contained test runner (HydroFlowTestSuite.java) validates boundary conditions and safety interlocks without third-party testing frameworks. The test suite executes six verification cases:

Test ID	Test Target	Input Condition & Assertion	Result
TC-01	Asset Identifier Regex Validation	Validate compliant IDs; assert InvalidNodeIdentifierException thrown on special characters.	PASSED
TC-02	Polymorphic Pump Power Dispatch	Calculate power at 40 L/s @ 35m head; assert VFD power < fixed-speed pump power.	PASSED
TC-03	Reservoir Reserve Interlock	Attempt discharge dropping storage below 15%; assert ReservoirDepletionException thrown.	PASSED
TC-04	Joukowsky Surge Protection	Simulate rapid valve closure creating 19.57 bar surge; assert PipeBurstSurgeException thrown.	PASSED
TC-05	Multithreaded Telemetry Worker	Spawn sensor Runnable thread; assert background telemetry packets populated after join.	PASSED
TC-06	CSV Stream File Persistence	Serialize reservoir list to CSV; read back via BufferedReader and assert record integrity.	PASSED

12.1 Boundary Value Analysis Table

Boundary Parameter	Minimum / Edge	Nominal Value	Observed System Behavior
Pipeline Flow Rate	0.0 L/s	85.0 L/s	Zero flow returns 0.0 m head loss; nominal flow calculates 2.107 m loss.
Reservoir Reserve Ratio	15.0% (Exact Reserve)	84.0%	Discharge permitted down to 15.0%; breach blocked with custom exception.
Pipe Peak Surge Pressure	6.2 bar (Static)	19.57 bar (Surge)	Surge exceeding 16.0 bar PN rating triggers emergency bypass trip.
VFD Operational Frequency	30.0 Hz	50.0 Hz	Frequency clamped between 30.0 Hz and 60.0 Hz to prevent motor overheating.

13. PRACTICAL CHALLENGES FACED & SOLUTIONS

During implementation, four technical hurdles required structured resolution. First, when simulated pump stations engaged multiple booster units concurrently, worker threads competed to update cumulative transformer power draws; adding synchronized monitor locks on `PumpingStation` instances and making power allocation checks atomic resolved the race condition. Second, the initial asset regular expression strictly matched ISO alphanumeric tags, rejecting valid zone hyphenated strings; adjusting the pattern to `^[A-Z0-9]{6,16}$` accommodated uppercase asset codes while preventing SQL-like injection characters and whitespace.

Third, standard JUnit testing requires build tool wrappers such as Maven or Gradle, which introduces environment dependency risks during lab evaluation; writing a standalone test suite in `HydroFlowTestSuite.java` using native Java assertions (`-ea`) kept the build portable across JDK installations. Fourth, writing persistent reservoir CSV records and telemetry logs concurrently risked descriptor leakage during abnormal exits, which was resolved by wrapping all `FileWriter` and `BufferedWriter` operations in `try-with-resources` blocks.

14. CORE LEARNINGS & ACADEMIC TAKEAWAYS

The development of HydroFlow OS reinforced core syllabus topics across Units 1 through 5 of the Programming in Java curriculum:

- Unit 1 covers control flow and primitive data handling through interactive menu navigation loops, double-precision head loss calculations, and formatted console diagnostics.
- Unit 2 principles appear in the network node class hierarchy, parameterized constructors invoking `super`, access encapsulation, and runtime dynamic method dispatch.
- Unit 3 concepts are realized through abstract methods in `Pump.java` and decoupled interface contracts (`HydraulicActuator`, `PressureRegulator`, `Auditable`).
- Unit 4 techniques govern error recovery through a checked exception hierarchy rooted in `HydroFlowException`, alongside asynchronous sensor telemetry managed via the `Runnable` interface.
- Unit 5 collection and I/O structures manage grid state, including 2D arrays for shift schedules, `LinkedHashMap` for $O(1)$ asset indexing, a `PriorityQueue` for reservoir refilling priority, and `BufferedReader` and `BufferedWriter` streams for audit persistence.

15. FUTURE ENHANCEMENTS

- Integrating time-of-use electrical tariffs to shift major pumping schedules into lowest-cost nighttime hours.
- Adding real-time acoustic transient modeling to detect small distribution leaks before underground pipe bursts occur.
- Connecting the simulated pipeline to physical microcontrollers (such as ESP32 boards) to stream real-world pressure transducer readings via cellular telemetry.

16. REFERENCES & ACADEMIC CITATIONS

1. Bloch, J. (2018). *Effective Java (3rd Edition)*. Addison-Wesley Professional.
2. Schildt, H. (2021). *Java: The Complete Reference (12th Edition)*. McGraw-Hill Education.
3. Walski, T. M., Chase, D. V., Savic, D. A., Grayman, W., Beckwith, S., & Koelle, E. (2003). *Advanced Water Distribution Modeling and Management*. Haestad Press.
4. Joukowsky, N. (1898). *Water Hammer*. Translated by O. Simin (1904), Proceedings of the American Water Works Association, 24, 341-424.
5. Oracle Corporation. (2024). *Java Platform, Standard Edition Documentation (Java SE 17 & 21)*. Oracle Technology Network.